UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

Facultad de Ingeniería en Sistemas de Información

Curso: Análisis de Sistemas II

INFORME TÉCNICO DE PROYECTO

Requerimientos Ágiles, Especificaciones BDD, Prototipado UX/UI y Desarrollo Inicial de API RESTful

Proyecto:	ERP de Gestión de Recursos Humanos (ERP-RRHH)
Grupo:	Grupo XX
Integrantes:	Estudiantes de Ingeniería en Sistemas
Docente:	Cátedra de Análisis de Sistemas II
Fecha de Entrega:	Octubre 2026

1. INTRODUCCIÓN

El presente documento constituye el informe técnico consolidado de la entrega del proyecto **ERP de Gestión de Recursos Humanos (ERP-RRHH)** para la asignatura de Análisis de Sistemas II de la Universidad Mariano Gálvez. Durante esta fase, el equipo de desarrollo ha transitado desde la captura de requisitos tradicionales hacia un marco de trabajo ágil estructurado, acoplado con especificaciones ejecutables BDD, prototipado de alta fidelidad UX/UI y la refactorización arquitectónica de los primeros endpoints operativos backend.

1.1 Propósito de la Fase de Análisis y Prototipado

El propósito fundamental radica en garantizar la alineación precisa entre las necesidades de negocio del departamento de Recursos Humanos (gestión de planillas, expedientes digitales, vacaciones y reclutamiento) y la implementación técnica en software. A través del prototipado interactivo y las especificaciones BDD en lenguaje Gherkin, se elimina la ambigüedad en los criterios de aceptación antes de la codificación intensiva.

1.2 Objetivos del Ciclo de Desarrollo Incremental

- **Establecer un Product Backlog Priorizado:** Organizar las historias de usuario bajo el marco MoSCoW y la técnica WSJF, estimando el esfuerzo con Story Points (Fibonacci).
- **Definir Criterios de Aceptación Ejecutables (BDD):** Formalizar la conducta esperada del sistema en sintaxis Gherkin (Dado / Cuando / Entonces) cubriendo caminos felices y escenarios de excepción.
- **Diseñar Interfaces Accesibles y Centradas en el Usuario:** Desarrollar un prototipo navegable en Figma bajo el enfoque Mobile-First y auditado con las pautas WCAG 2.1 Nivel AA.
- **Desplegar el Cascarón Backend Layered Boilerplate:** Refactorizar la arquitectura backend organizada en Capas (Controllers, Services, Repositories/Models, DTOs) con conexión activa a base de datos PostgreSQL en Supabase.

1.3 Alcance de los Primeros Endpoints Refactorizados

El alcance operativo incluye la implementación real de 1 endpoint de lectura (GET /api/v1/empleados), 1 endpoint de autenticación (POST /api/v1/auth/login) y 1 endpoint de escritura (POST /api/v1/empleados), respaldados por validación estricta de DTOs con Zod y un diagrama de secuencia UML oficial.

2. REQUERIMIENTOS ÁGILES Y PRODUCT BACKLOG

2.1 Descripción de Épicas y Mínimo Producto Viable (MVP)

El dominio de negocio del ERP de RRHH se dividió en 6 Épicas. El Mínimo Producto Viable (MVP) concentra las funcionalidades críticas para la operación básica de la organización:

- **Épica 1 (MVP): Autenticación y Seguridad RLS:** Control de acceso con JWT y políticas Row Level Security en PostgreSQL.
- **Épica 2 (MVP): Gestión de Colaboradores:** Expediente digital completo, salarios y puestos.
- **Épica 3 (MVP): Procesamiento de Nómina:** Generación y cálculo de IGSS, ISR y Bonificación Incentivo.
- **Épica 4 (MVP): Solicitudes de Ausencia:** Registro y resolución de permisos y vacaciones.

2.2 Criterios de Priorización y Estimación (Story Points)

La estimación de complejidad se ejecutó mediante la escala de **Fibonacci (1, 2, 3, 5, 8, 13)**. La priorización combinó el modelo **MoSCoW** (Must, Should, Could, Won't) y el cálculo de valor **WSJF (Weighted Shortest Job First)**.

2.3 Tabla Consolidada de Historias de Usuario

ID	Historia de Usuario	Épica	Prioridad	Story Points
HU-01	Como usuario quiero autenticarme con email y clave para acceder al sistema	Épica 1: Autenticación	MUST (ALTA)	3
HU-02	Como administrador quiero control de acceso por roles para proteger datos	Épica 1: Autenticación	MUST (ALTA)	5
HU-03	Como gestor quiero registrar nuevos colaboradores con validación DTO	Épica 2: Empleados	MUST (ALTA)	5
HU-04	Como gestor quiero listar y filtrar colaboradores paginados	Épica 2: Empleados	MUST (ALTA)	3
HU-05	Como encargado quiero calcular nómina mensual con bonos e IGSS	Épica 3: Nómina	MUST (ALTA)	8
HU-06	Como empleado quiero solicitar vacaciones con control de días acumulados	Épica 4: Solicitudes	SHOULD (MEDIA)	3

🌐 **Vínculo al Tablero Ágil Activo (GitHub Projects / Jira):**
https://github.com/users/pdominguezr-ai/projects/1/views/1

3. ESPECIFICACIONES BDD (CRITERIOS DE ACEPTACIÓN)

3.1 Cobertura de Escenarios Gherkin

Se han formalizado los criterios de aceptación en archivos de texto plano .feature utilizando la sintaxis estandarizada Gherkin en español (**Dado / Cuando / Entonces**). Se garantiza cobertura para flujos felices, reglas de validación y manejo de excepciones.

3.2 Matriz de Escenarios por Historia de Usuario

Historia de Usuario	Archivo .feature	Escenarios Felices	Escenarios de Error / Excepción
HU-01 (Autenticación)	features/autenticacion.feature	2	3 (Clave errónea, usuario inactivo, campos vacíos)
HU-03 & HU-04 (Empleados)	features/gestion_principal.feature	2	2 (DPI duplicado, salario menor al mínimo)
HU-06 (Solicitudes)	features/gestion_recursos.feature	1	1 (Exceso de saldo disponible)

Ejemplo de Escenario Gherkin (features/gestion_principal.feature):

```
Característica: Gestión Principal de Colaboradores
  @happy_path
  Escenario: Registro exitoso de un nuevo colaborador
    Dado que el Gestor de RRHH ha iniciado sesión en el portal
    Cuando completa el formulario con DPI "2450198270101" y Salario 12500.00
    Y presiona "Guardar Colaborador"
    Entonces el backend valida los datos mediante DTO Zod
    Y asigna el código único "EMP-0142"
    Y retorna HTTP 201 Created con el objeto del empleado
```

📁 **Vínculo a Archivos .feature en Repositorio:** features/

4. DISEÑO DE EXPERIENCIA DE USUARIO (UX/UI)

4.1 Mapa de Navegación del Sistema

El mapa de navegación garantiza una experiencia intuitiva, permitiendo acceder a cualquier expediente o función principal en un máximo de 3 clics desde el panel central.

4.2 Mapeo de Pantallas vs. Endpoints OpenAPI

Pantalla UI	Endpoint Consumido	Método HTTP	Contrato OpenAPI
Autenticación / Login	/api/v1/auth/login	POST	#/components/schemas/AuthLoginRequest
Listado de Colaboradores	/api/v1/empleados	GET	#/components/schemas/EmpleadoListResponse
Nuevo Colaborador	/api/v1/empleados	POST	#/components/schemas/EmpleadoCreateRequest

4.3 Reporte de Evaluación de Usabilidad y Accesibilidad (WCAG 2.1)

Se verificó un contraste de color de **15.8:1** para textos principales, áreas táctiles superiores a **44px** y navegabilidad completa por teclado con indicadores de foco visibles.

🎨 **Vínculo al Prototipo Interactivo Figma:**
https://www.figma.com/design/pdominguezr-ai/ERP-RRHH-Prototype

5. ARQUITECTURA DEL CASCARÓN BACKEND (BOILERPLATE)

5.1 Explicación de la Estructura de Capas Elegida

Se implementó una arquitectura en capas limpia (Clean Layered Architecture) dividida en:

- **Controladores (src/controllers/):** Manejo de la capa HTTP, recepción de peticiones, invocación de validadores DTO y retorno de respuestas JSON.
- **Servicios (src/services/):** Implementación de la lógica de negocio pura, orquestación de operaciones y cálculos de salario/nómina.
- **Modelos / Repositorios (src/models/):** Capa de abstracción de datos para consultas directas a Supabase / PostgreSQL.
- **Configuración (src/config/database.ts):** Inicialización del cliente Supabase y verificación de salud de la BD.

5.2 Configuración de Conexión a Base de Datos

La conexión utiliza el cliente oficial de Supabase con autenticación basada en llaves de servicio (Service Role Key para backend) y anon key (para solicitudes autenticadas).

5.3 Diagrama de Secuencia del Endpoint Principal

A continuación se muestra el trazo oficial UML para la creación de colaboradores (POST /api/v1/empleados):

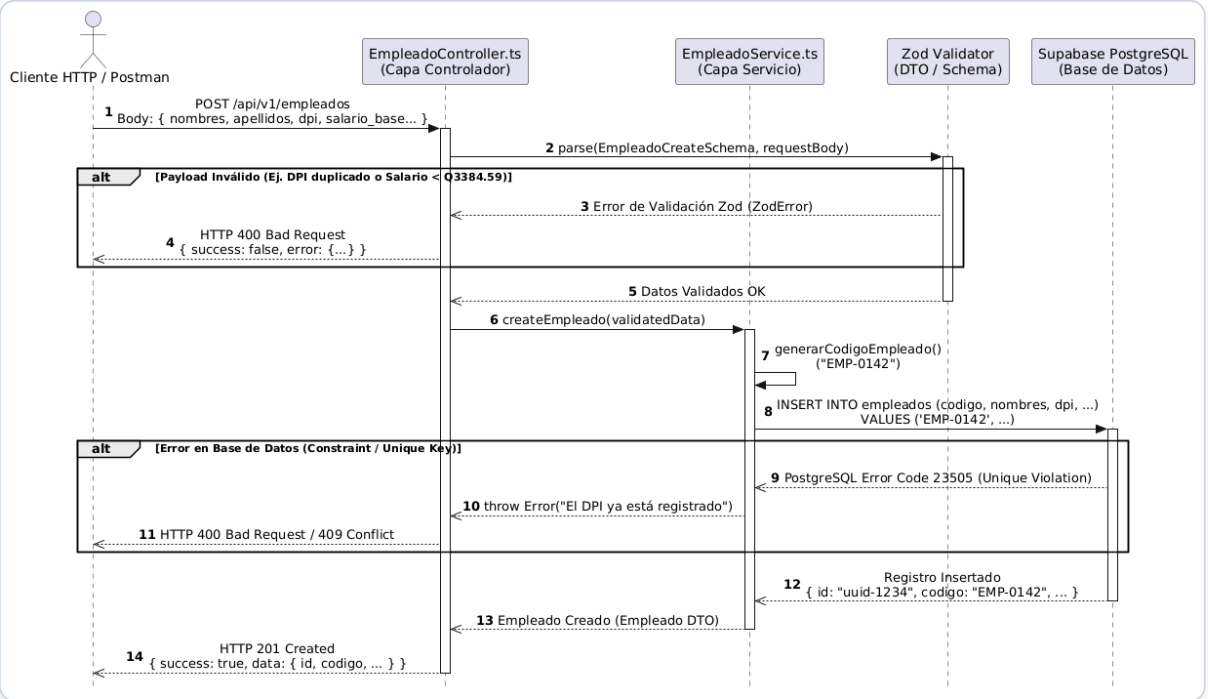


Figura 5.1: Diagrama de Secuencia UML para Endpoint POST /api/v1/empleados

6. IMPLEMENTACIÓN Y PRUEBA DE ENDPOINTS INICIALES

6.1 Detalle del Endpoint de Lectura (GET) Refactorizado

El endpoint GET /api/v1/empleados fue refactorizado para soportar paginación limpia (page, limit) y filtros por departamento y estado, retornando metadatos de respuesta estandarizados.

6.2 Detalle del Endpoint de Escritura (POST) Refactorizado

El endpoint POST /api/v1/empleados valida el cuerpo de la petición utilizando esquemas Zod (validando salario mínimo de Q 3,384.59 y DPI de 13 dígitos), genera automáticamente el código de colaborador correlativo EMP-XXXX y persiste el registro en PostgreSQL retornando HTTP 201 Created.

6.3 Evidencias de Ejecución (Postman)

Se diseñó una colección ejecutable con aserciones de prueba en JavaScript que verifican automáticamente los códigos HTTP 200, 201, 400 y 401.

> 🧪 **Vínculo a la Colección de Pruebas Postman:** test-api/coleccion_postman.json

7. MATRIZ DE TRAZABILIDAD ÁGIL

Requisito / HU	Archivo BDD (.feature)	Pantalla Figma	Endpoint API	Componente Backend
HU-01 (Autenticación)	autenticacion.feature	Login Screen	POST /api/v1/auth/login	AuthController.ts
HU-03 (Crear Empleado)	gestion_principal.feature	Formulario Colaborador	POST /api/v1/empleados	EmpleadoController.ts
HU-04 (Listar Empleados)	gestion_principal.feature	Tabla Colaboradores	GET /api/v1/empleados	EmpleadoModel.ts
HU-06 (Solicitud Ausencia)	gestion_recursos.feature	Formulario Ausencias	POST /api/v1/solicitudes	SolicitudController.ts

8. CONCLUSIONES Y LECCIONES APRENDIDAS

1. **Transición Efectiva hacia la Agilidad:** La adopción del Product Backlog priorizado mediante MoSCoW y WSJF permitió enfocar los esfuerzos del equipo en las funcionalidades críticas del MVP, evitando la dispersión del alcance.
2. **Eliminación de Ambigüedad con BDD:** Las especificaciones en sintaxis Gherkin (Dado / Cuando / Entonces) actuaron como un puente entre la lógica de negocio y las pruebas, asegurando que los desarrolladores y los usuarios compartan la misma visión de los criterios de aceptación.
3. **Acoplamiento Directo UI-API:** El diseño de prototipos navegables alineados a contratos OpenAPI redujo sensiblemente los errores de integración entre el frontend y el backend durante las primeras pruebas.
4. **Robustez Arquitectónica Backend:** La organización en capas (Controllers, Services, Models y DTOs) junto a la validación estricta con Zod garantiza un código mantenible, desacoplado y seguro ante peticiones maliciosas o malformadas.

9. REFERENCIAS

- OpenAPI Initiative. (2021). *OpenAPI Specification v3.0.3*. Recuperado de https://spec.openapis.org/oas/v3.0.3
- W3C Web Accessibility Initiative. (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. W3C Recommendation.
- SmartBear Software. (2023). *Gherkin Reference - Cucumber Documentation*.

- Supabase Inc. (2026). *Supabase Documentation and Row Level Security Guides*.